

Todo lo que entra al parcial

BFS · Dijkstra · Bellman-Ford · Union-Find · Kruskal · Prim
Con todos los patrones, variantes y ejercicios resueltos

01	BFS y variantes	grilla · multi-source · 0-1 · reconstrucción
02	Dijkstra y variantes	estados · invertido · minimax
03	Bellman-Ford	negativos · ciclos · maximizar
04	Union-Find y MST	Kruskal · Prim · minimax · componentes
05	Tabla de decisión	cuándo usar cada uno
06	Errores comunes	lo que cuesta el parcial
07	Mapa de ejercicios	todos los resueltos

Ante la duda: si el enunciado dice "mínimo costo para conectar todo" → MST. Si dice "camino más corto entre dos nodos" → BFS/Dijkstra/BF según los pesos.

BFS y variantes

Camino más corto en cantidad de aristas · grillas · múltiples orígenes

Cola FIFO. La primera vez que tocás un nodo, su distancia ya es la óptima. **dist[x] = -1** hace de "no visitado" y distancia a la vez.

BFS base

```
from collections import deque
import sys; input = sys.stdin.readline

def bfs(grafo, origen, n):
    dist = [-1] * (n + 1)
    dist[origen] = 0
    cola = deque([origen])
    while cola:
        nodo = cola.popleft()
        for vecino in grafo[nodo]:
            if dist[vecino] == -1:
                dist[vecino] = dist[nodo] + 1
                cola.append(vecino)
    return dist
```

BFS con reconstrucción de camino

```
def bfs_camino(grafo, origen, destino):
    n = len(grafo)
    dist = [-1] * n
    padre = [-1] * n
    dist[origen] = 0
    cola = deque([origen])
    while cola:
        nodo = cola.popleft()
        for vecino in grafo[nodo]:
            if dist[vecino] == -1:
                dist[vecino] = dist[nodo] + 1
                padre[vecino] = nodo
                cola.append(vecino)
    if dist[destino] == -1: return None
    camino, actual = [], destino
    while actual != -1:
        camino.append(actual); actual = padre[actual]
    camino.reverse()
    return camino
```

Multi-source BFS

Cuando querés la distancia al **más cercano de un conjunto** de orígenes. Metés todos en la cola a distancia 0. El BFS se expande desde todos simultáneamente.

```
# arrancás la cola con TODOS los orígenes en distancia 0
cola = deque()
for mi, mj in orígenes:
    dist[mi][mj] = 0
    cola.append((mi, mj))
# después el BFS corre igual que siempre
```

CSES 1194 — Monster (distancia al monstruo más cercano)

BFS en grilla

```
DIRS = [(-1,0,'U'), (1,0,'D'), (0,-1,'L'), (0,1,'R')]

def bfs_grilla(grid, inicio, n, m):
    dist = [[-1]*m for _ in range(n)]
    padre = [[None]*m for _ in range(n)]
    si, sj = inicio
    dist[si][sj] = 0
    cola = deque([(si, sj)])
    while cola:
        i, j = cola.popleft()
        for di, dj, c in DIRS:
            ni, nj = i + di, j + dj
            if 0 <= ni < n and 0 <= nj < m \
               and grid[ni][nj] != '#' and dist[ni][nj] == -1:
                dist[ni][nj] = dist[i][j] + 1
                padre[ni][nj] = (i, j, c)
                cola.append((ni, nj))
    return dist, padre
```

MONSTER · LA CONDICIÓN CLAVE

El jugador solo puede pisar una celda si llega **estrictamente antes** que el monstruo más cercano:

`dist_jugador[i][j] + 1 < dist_monstruo[ni][nj]`. El `<` es estricto — empatar significa que el monstruo te agarra.

Dijkstra 0-1

Cuando los pesos son solo 0 o 1. Usás **deque** en vez de heap: peso 0 → `appendleft`, peso 1 → `append`.

```
def dijkstra_01(grafo, origen, n):
    INF = float('inf')
    dist = [INF] * (n + 1)
    dist[origen] = 0
    dq = deque([(0, origen)])
    while dq:
        d, nodo = dq.popleft()
        if d > dist[nodo]: continue # lazy deletion
        for vecino, peso in grafo[nodo]:
            nd = d + peso
            if nd < dist[vecino]:
                dist[vecino] = nd
                if peso == 0: dq.appendleft((nd, vecino))
                else: dq.append((nd, vecino))
    return dist
```

UVA 11573 — Ocean Currents

Dijkstra y variantes

Camino más corto con pesos no negativos · greedy con heap

Min-heap con (distancia, nodo). Primera vez que sale un nodo = distancia definitiva.

Lazy deletion: si $d > \text{dist}[u]$, la entrada es vieja, la salteás.

Template base

```
import heapq
INF = float("inf")

def dijkstra(G, src):
    dist = [INF] * len(G)
    q = [(0, src)]
    while q:
        d, u = heapq.heappop(q)
        if dist[u] < INF: continue # lazy deletion
        dist[u] = d
        for w, v in G[u]:
            if dist[v] == INF:
                heapq.heappush(q, (d + w, v))
    return dist
```

CSES 1671 — Shortest Routes I

OJO · ARISTAS COMO (PESO, VECINO)

El heap ordena por el primer elemento, así que el peso va primero: `G[u].append((peso, vecino))`. Si lo ponés al revés, Dijkstra ordena por número de nodo y da resultados incorrectos.

Variante · invertido (todos hacia un destino)

Si querés la distancia de **todos los nodos hacia un destino fijo**, invertí las aristas y corré un solo Dijkstra desde el destino.

```
for _ in range(m):
    u, v, w = map(int, input().split())
    G[v].append((w, u)) # arista al REVÉS
res = dijkstra(G, destino)
```

UVA 1112 — Mice and Maze

Variante · estados (condición extra)

Cuando el nodo lleva info extra (cupón, pared rota, etc.). El estado es `(nodo, info)` y `dist` pasa a ser una matriz 2D.

```

# dist[nodo][0] = sin usar cupón, dist[nodo][1] = cupón usado
dist = [[INF] * 2 for _ in range(n + 1)]
dist[origen][0] = 0
heap = [(0, origen, 0)]    # (dist, nodo, cupón_usado)
while heap:
    d, nodo, usado = heapq.heappop(heap)
    if d > dist[nodo][usado]: continue
    for vecino, peso in grafo[nodo]:
        if d + peso < dist[vecino][usado]:
            dist[vecino][usado] = d + peso
            heapq.heappush(heap, (d + peso, vecino, usado))
        if usado == 0 and d + peso//2 < dist[vecino][1]:
            dist[vecino][1] = d + peso//2
            heapq.heappush(heap, (d + peso//2, vecino, 1))

```

CSES 1195 — Flight Discount

Bellman-Ford

Pesos negativos · detección y reconstrucción de ciclos

Relaja **todas** las aristas $N-1$ veces. La ronda N de control detecta ciclos. Para Bellman-Ford, guardá las aristas como **lista plana** (u, v, w) , no lista de adyacencia.

MAXIMIZAR VS MINIMIZAR

Bellman-Ford también sirve para el camino de **mayor** peso (maximizar). Solo cambiás MINF en vez de INF, la comparación al revés ($>$ en vez de $<$), y buscás ciclos **positivos** en vez de negativos.

Detección de ciclo negativo + reconstrucción

```
INF = float('inf')

def find_negative_cycle(edges, n):
    dist = [0] * (n + 1)  # 0 para detectar cualquier ciclo
    padre = [-1] * (n + 1)
    x = -1
    for i in range(n):
        x = -1
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                padre[v] = u
                x = v
        if x == -1: break  # convergió, no hay ciclo
    if x == -1: return None
    for _ in range(n):  # remontar N veces = entrar al ciclo
        x = padre[x]
    ciclo, nodo = [], x
    while True:
        ciclo.append(nodo)
        nodo = padre[nodo]
        if nodo == x:
            ciclo.append(nodo); break
    ciclo.reverse()
    return ciclo
```

CSES 1197 — Cycle Finding

Maximizar + detectar ciclo positivo (High Score)

```
MINF = float('-inf')

def dists_from(G, src, n):
    dist = [MINF] * (n + 1); dist[src] = 0
    process = [src]; changed = [False] * (n + 1)
    for _ in range(n):
        if not process: break
        for w in process: changed[w] = False
        prev, process = process, []
        for v in prev:
            for w, d in G[v]:
                if dist[w] < dist[v] + d:
                    dist[w] = dist[v] + d
                    if not changed[w]: process.append(w)
                    changed[w] = True
    return dist, changed

# En solve: correr F(desde 1) y B(desde n con aristas invertidas)
# Si changed_f[v] AND dist_b[v] != MINF → ciclo positivo en el camino → -1
```

CSES 1673 — High Score

Union-Find · Kruskal · Prim

Conectar todos los nodos con mínimo costo total · N-1 aristas · sin ciclos

Un MST necesita exactamente **N-1 aristas**. Si al terminar Kruskal usaste menos, el grafo es desconexo → IMPOSSIBLE. Kruskal por defecto; Prim para grafos muy densos.

Union-Find (DSU)

```
import sys; sys.setrecursionlimit(300000)

class UF:
    def __init__(self, n):
        self.p = list(range(n))    # cada uno es su propia raíz
        self.s = [1] * n          # tamaño de cada grupo
    def find(self, v):
        if self.p[v] != v:
            self.p[v] = self.find(self.p[v])    # path compression
        return self.p[v]
    def unite(self, v, w):
        rv, rw = self.find(v), self.find(w)
        if rv == rw: return False
        if self.s[rv] < self.s[rw]: rv, rw = rw, rv    # union by size
        self.p[rw] = rv; self.s[rv] += self.s[rw]
        return True
```

TRUCO · UF CON N+1

Si los nodos van de 1 a n, creá el UF con **n+1** elementos e ignorá el 0. Te ahorrarás restar 1 en cada operación. El tamaño máximo después de unite: `uf.s[uf.find(v)]` (siempre consultás el tamaño desde la raíz).

Kruskal

```
def solve():
    n, m = map(int, input().split())
    E = []
    for _ in range(m):
        v, w, c = map(int, input().split())
        E.append((c, v, w))    # costo primero → sort ordena por costo
    E.sort()
    uf = UF(n + 1); res = 0; e = 0
    for c, v, w in E:
        if uf.unite(v, w):
            res += c; e += 1
            if e == n - 1: break    # MST completo, no hace falta seguir
    if e < n - 1: print("IMPOSSIBLE")
    else: print(res)
```

Kruskal + minimax (cuello de botella mínimo)

Guardás el árbol resultante como lista de adyacencia y respondés cada consulta con un BFS que lleva el máximo de pesos. La arista más cara del camino óptimo entre A y B en el MST es el cuello de botella mínimo.

```
# En Kruskal: guardar el árbol
mst = [[] for _ in range(n + 1)]
if uf.unite(u, v):
    mst[u].append((v, w)); mst[v].append((u, w))

# Para cada consulta (a, b): BFS sobre el árbol llevando el max
def max_en_camino(mst, origen, destino, n):
    if origen == destino: return 0
    visitado = [False] * (n + 1)
    max_hasta = [0] * (n + 1)
    visitado[origen] = True
    cola = deque([origen])
    while cola:
        u = cola.popleft()
        if u == destino: return max_hasta[u]
        for v, peso in mst[u]:
            if not visitado[v]:
                visitado[v] = True
                max_hasta[v] = max(max_hasta[u], peso)
                cola.append(v)
    return float('inf') # no hay camino
```

UVA 10048 — Audiophobia · UVA 1216 — Bug Sensor · UVA 10099 — Tourist Guide

Union-Find dinámico (contar componentes)

```
number_components = n
max_size = 1
for v, w in aristas:
    if uf.find(v) != uf.find(w):
        uf.unite(v, w)
        number_components -= 1
        max_size = max(max_size, uf.s[uf.find(v)])
print(number_components, max_size)
```

CSES 1676 — Road Construction

Prim

```
def prim(grafo, n):
    visitado = [False] * (n + 1)
    heap = [(0, 1)] # (peso, nodo) - peso de la arista, no distancia acumulada
    total, usados = 0, 0
    while heap and usados < n:
        peso, u = heapq.heappop(heap)
        if visitado[u]: continue
        visitado[u] = True; total += peso; usados += 1
        for w, v in grafo[u]:
            if not visitado[v]: heapq.heappush(heap, (w, v))
    return total if usados == n else None
```

PRIM VS DIJKSTRA · LA ÚNICA DIFERENCIA

En Dijkstra pusheás `dist[u] + peso` (distancia acumulada). En Prim pusheás solo `peso` (el costo de esa arista). A Prim no le importa qué tan lejos del inicio estás, solo cuánto cuesta agregar ese nodo al árbol.

¿Cuál usar?

Tabla de referencia rápida

Pista del enunciado	Algoritmo
Sin pesos, camino más corto	BFS
Distancia al más cercano de varios orígenes	Multi-source BFS
Grilla con costo 0/1	Dijkstra 0-1 (deque)
Pesos ≥ 0	Dijkstra
Pesos negativos / ciclos negativos	Bellman-Ford
Maximizar score / ciclos positivos	Bellman-Ford (MINF, doble pasada)
Todos hacia un destino	Dijkstra con aristas invertidas
Condición extra en el camino (cupón, etc.)	Dijkstra con estados
Minimizar la arista más cara del camino	Kruskal + BFS minimax
Conectar todo al mínimo costo total	Kruskal
¿Están conectados? / cuántos grupos	Union-Find puro
Agregar aristas y reportar componentes	Union-Find dinámico

06

Errores comunes

- ▶ **Peso antes que vecino:** guardá siempre `(peso, vecino)`, no `(vecino, peso)`. Heap y sort ordenan por el primer elemento.
- ▶ **INF muy chico:** si los pesos llegan a 10^9 , `1 < 28` no alcanza. Usá `float('inf')`.
- ▶ **Olvidar input rápido:** `input = sys.stdin.readline` o TLE.
- ▶ **Dijkstra con pesos negativos:** no funciona. Usá Bellman-Ford.
- ▶ **Bellman-Ford sin `dist[u] != INF`:** propagás basura desde nodos inalcanzables.
- ▶ **Bellman-Ford en lista de adyacencia:** más lento que lista plana `(u, v, w)`. Usá lista plana.
- ▶ **find recursivo sin `setrecursionlimit`:** RecursionError en grafos grandes. Agregá `sys.setrecursionlimit(300000)`.
- ▶ **UF: `s[v]` vs `s[find(v)]`:** solo la raíz tiene el tamaño correcto. Siempre `s[uf.find(v)]`.
- ▶ **Kruskal: costo no va primero:** la tupla debe ser `(costo, u, v)` para que sort ordene por costo.
- ▶ **Imprimir nodo 0:** si el grafo es $1..n$ y el array tiene $n+1$ posiciones, acordate del `[1:]`.

- ▶ **UVA líneas en blanco:** usá una función `leer()` que saltea líneas vacías, o leé todo con `sys.stdin.read().split()`.
- ▶ **Monster · BFS en grilla grande:** con 1000×1000 celdas, Python puede dar TLE. El algoritmo es correcto; el cuello es la constante del lenguaje.

Mapa de ejercicios resueltos

Problema	Patrón	Dif.
UVA 1148 — Mysterious X Network	BFS básico · input UVA	1
CSES 1667 — Message Routes	BFS + reconstrucción	1
CSES 1194 — Monster	Multi-source BFS + escape en grilla	3
UVA 11573 — Ocean Currents	Dijkstra 0-1 en grilla	2
CSES 1671 — Shortest Routes I	Dijkstra base	1
UVA 1112 — Mice and Maze	Dijkstra invertido	1
CSES 1195 — Flight Discount	Dijkstra con estados	2
CSES 1673 — High Score	Bellman-Ford maximizar + ciclos positivos	1
CSES 1197 — Cycle Finding	Bellman-Ford + reconstrucción ciclo	2
UVA 793 — Network Connections	Union-Find puro	1
CSES 1675 — Road Reparation	Kruskal + IMPOSSIBLE	1
CSES 1676 — Road Construction	Union-Find dinámico + max_size	1
UVA 10048 — Audiophobia	Kruskal + BFS minimax	2